CloudNativePG: A Technical and Operational Analysis of a Kubernetes-Native PostgreSQL Operator

August 2026

Abstract

CloudNativePG (CNPG) is a Kubernetes operator for managing PostgreSQL clusters throughout their lifecycle. It uses a declarative Cluster custom resource, Kubernetes reconciliation, PostgreSQL native streaming replication, and an in-container instance manager to provide provisioning, self-healing, failover, backups, and upgrades. This paper examines CNPG's architecture and operating model, focusing on elasticity, failure handling, storage, performance, security, observability, and disaster recovery. It also distinguishes CNPG from StatefulSet-based deployments and from operators that depend on an external high-availability layer. Particular attention is given to the mechanisms that implement elasticity: the instance-join path, the three distinct mechanisms that transport write-ahead log during a topology change, the rolling-update path used for vertical changes, and the delegation of volume growth to the Container Storage Interface (CSI). The paper shows that the achievable elasticity and durability of a deployment are determined less by the operator's configuration than by two external choices: the failure domains available to it, and the storage backend beneath it. CNPG reduces the number of components involved in PostgreSQL failover, but it does not remove the need for reliable Kubernetes control-plane infrastructure, suitable storage, tested application reconnection behavior, or operational runbooks. CNPG is most suitable for teams that already operate Kubernetes and are prepared to treat database reliability as a property of both the data plane and the control plane.

Keywords: CloudNativePG, PostgreSQL, Kubernetes, database operators, high availability, failover, replication, disaster recovery

1 Introduction

Running PostgreSQL in Kubernetes requires more than placing a database process in a Pod. PostgreSQL has durable state, a primary and standby relationship, write-ahead logging (WAL), replication, client routing, and recovery procedures. Kubernetes supplies resource scheduling, storage abstractions, service discovery, and reconciliation, but it does not by itself understand PostgreSQL's transaction state. An operator connects these two models.

CloudNativePG is a Kubernetes-native PostgreSQL operator designed to manage a highly available primary/standby cluster using PostgreSQL native streaming replication [1, 2]. The desired state is expressed through the CNPG Cluster custom resource (CR). The operator observes that resource and reconciles Pods, PersistentVolumeClaims (PVCs), Services, certificates, and PostgreSQL configuration. The Kubernetes API server therefore acts as the authoritative interface for the cluster's declared topology.

CNPG does not use a StatefulSet to manage PostgreSQL instances. Instead, it directly manages Pods and PVCs. This gives the operator freedom to promote the most suitable standby rather than treating a Pod ordinal as a database identity [2, 3]. Each PostgreSQL Pod contains an instance manager that runs as the parent process and supervises PostgreSQL startup, shutdown, health, and replication-related behavior.

This paper analyzes the model described in CNPG's documentation and the operational consequences of using it. It does not treat claims about Crunchy PostgreSQL for Kubernetes as CNPG features. The distinction is necessary because both projects manage PostgreSQL on Kubernetes but expose different custom resources, commands, images, and lifecycle mechanisms.

2 Methodology and scope

This literature-based technical study is not a benchmark. Its primary sources are the CloudNativePG documentation and project repository, supplemented by Kubernetes and PostgreSQL documentation. It addresses four questions:

1. How does CNPG divide responsibility between Kubernetes, the operator, the instance manager, and PostgreSQL?
2. What operational model does CNPG use for scaling, failover, upgrades, and recovery?
3. Which performance, storage, security, and observability decisions remain the user's responsibility?
4. What trade-offs result from relying on the Kubernetes control plane instead of a separate failover manager?

The supplied source material contains repeated passages and several cross-project references. This paper retains operational concerns that apply to CNPG, but does not attribute other projects' features to it. In particular, the 'PostgresCluster' resource, 'pgo' commands, and Crunchy Data's default pgBackRest configuration are not CNPG behavior.

3 System model

3.1 Control plane and reconciliation

The CNPG operator implements a control loop. A user declares a cluster, including its PostgreSQL image, instance count, storage, replication, backup, and scheduling settings. The operator observes the CR and related Kubernetes resources, compares the desired and actual states, and acts on any difference. Status conditions and events show the result.

This model differs from an imperative runbook in which an administrator executes a fixed sequence of commands and assumes that the result will remain valid. A declarative resource can be reviewed, stored in version control, and reapplied. Reconciliation also supports self-healing after a Pod or node failure. It does not make every change risk free. Storage attachment, scheduling, replication lag, and API availability can delay convergence.

3.2 Main components

The main components are:

- **Cluster custom resource.** This resource describes the intended PostgreSQL cluster and is the principal user-facing configuration object.
- **Operator.** The operator watches the CR and Kubernetes resources, creates and updates workload resources, coordinates lifecycle actions, and publishes cluster status.
- **Instance manager.** A Go process runs as PID 1 in the PostgreSQL container. It manages the PostgreSQL process, probes, shutdown, startup, and instance-level health behavior [3].
- **PostgreSQL.** PostgreSQL provides the database engine, WAL, physical streaming replication, synchronous commit behavior, and recovery mechanisms.

- **Kubernetes resources.** Pods, PVCs, Services, Secrets, ConfigMaps, and certificates provide the execution, storage, routing, and identity primitives used by the cluster.

3.3 Pod and storage management

CNPG creates one Pod per PostgreSQL instance and manages the associated PVCs directly. This avoids the identity and ordering assumptions built into StatefulSets. Kubernetes StatefulSets remain useful for many stateful applications because they provide stable identities and ordered management [10]; CNPG bypasses them because database leadership is determined by PostgreSQL state rather than by an ordinal number.

The direct-management model makes storage policy important. A failed instance may require volume reattachment, a replacement PVC, or recovery from another replica or an object store. The operator can coordinate these steps, but it cannot make an unsuitable storage class durable, fast, or available across failure domains. Production designs should place replicas on separate nodes or availability zones and test the storage behavior expected during node loss.

4 Deployment and configuration

CNPG can be installed with standard Kubernetes tooling, including Helm and Operator Lifecycle Manager distributions where available. After installation, a Cluster CR defines the PostgreSQL workload. A minimal configuration can be concise. Production configurations usually add resource requests, storage, affinity, backup destinations, certificates, monitoring, and disruption policies.

The declarative model supports Infrastructure as Code and GitOps workflows. A configuration change should be made in the CR or in the source repository that generates it, rather than by changing a live Pod or using an unrelated scaling command. This keeps the declared and live instance counts together. If they diverge, the cluster may show confusing status, reconcile an unexpected state, or leave resources behind.

CNPG exposes separate Services for different traffic roles. The read-write service follows the current primary, while read-only services can expose eligible standbys. Applications should use these Services rather than Pod IP addresses. During a promotion, CNPG updates the read-write routing so that clients can locate the new primary [2].

5 Replication, elasticity, and scaling

5.1 Replication model

CNPG uses PostgreSQL physical streaming replication. A cluster normally has one primary and zero or more hot standby instances. Replication may be asynchronous or synchronous. Asynchronous replication permits the primary to continue without waiting for a standby, but a failure can lose transactions that had not reached a surviving replica. Synchronous replication reduces that risk according to the selected quorum and commit settings, at the cost of additional write latency and greater dependence on the availability of synchronous standbys.

The choice is easier to assess in terms of recovery objectives. Recovery point objective (RPO) describes the acceptable amount of data loss, while recovery time objective (RTO) describes the acceptable service interruption. A simple planning relationship is:

$$\text{RPO} \approx \text{unreplicated WAL} + \text{unarchived WAL}. \tag{1}$$

This is not a performance formula. It is a reminder that synchronous replication, asynchronous replication, and WAL archiving protect different failure scopes and should be evaluated together.

5.2 Horizontal scaling: the join path

The instance count in the Cluster resource is the only horizontal scaling control. CNPG exposes it through the Kubernetes scale subresource, which makes it appear reachable by a Horizontal Pod Autoscaler, but the project argues against that arrangement. A Cluster's selector covers the primary and all standbys, whose workload profiles are opposed, so a per-Pod average of a CPU or memory metric is not a meaningful control signal for either of them; adding a standby dilutes that average without relieving pressure on the write path [17]. Horizontal scale in CNPG is therefore a topology change rather than a capacity response, and it is better driven by an operator or a policy that understands read and write roles than by a resource metric.

When the count increases, the operator provisions a new PVC group and schedules a Pod whose data directory is empty. The instance manager on that Pod recognises the empty state, performs a physical base backup of the current primary over PostgreSQL's streaming replication protocol, and then configures the resulting data directory to stream from that primary. Since operator version 1.27 this join runs inside the instance manager rather than in a separate Kubernetes Job, which removes a scheduling dependency from the scale-out path.

The cost model of this path is the most consequential fact about CNPG elasticity. The join transfers a complete copy of the data directory. It is not incremental and it is not parallelised across several streams, so time-to-readiness is dominated by database size and link bandwidth rather than by the amount of data that has actually changed. This is the same asymmetry that motivates the live-migration literature, in which a background copy is combined with a bounded foreground delta so that downtime becomes independent of database size [33]. CNPG does not implement such a mechanism; its scale-out is a cold full copy followed by replay.

Scale-out also changes failure behaviour only with a delay. A new replica increases redundancy once it has caught up and is eligible for the intended service or promotion role, not when its Pod becomes Ready. A lagging replica may be useful as a recovery source while being unsuitable for read traffic or automatic promotion. Scale-in must account for synchronous replica requirements, read traffic, maintenance capacity, and the possibility that a claim is retained for recovery.

5.3 WAL transport during a topology change

Three distinct mechanisms carry write-ahead log between instances, and they protect different intervals. Distinguishing them is necessary because the phrase "WAL transfer" is often used as though a single mechanism were responsible.

The first operates during the join. The base backup is taken with streaming WAL, so a second walsender connection ships the log generated while the data files are copied. There is consequently no gap between the start LSN of the backup and the point at which the new instance can stream normally: the copy is a consistent snapshot taken in motion rather than a quiesced copy followed by a separate replay phase [22].

The second operates after the join and for the lifetime of the instance. CNPG maintains a cluster-managed physical replication slot per high-availability instance on the primary, identified by a configurable prefix whose default is `_cnpg_`. A slot prevents the primary from recycling log that a consumer has not acknowledged, so transient replica unavailability is converted from a risk of unrecoverable divergence into bounded growth of the primary's WAL directory. Slots are themselves synchronised to standbys at a configurable interval, thirty seconds by default, using PostgreSQL's slot-advance primitive, so that the retention guarantee survives a change of primary [14].

The third operates as a fallback. When continuous backup is configured, CNPG also points replicas at the WAL archive through the restore command, so a replica that has lost its streaming connection can recover segments from object storage rather than fail [14].

The combination has a sharp edge that belongs in any capacity plan. Because slot retention

is unbounded by default, a standby that stops consuming log indefinitely will eventually fill the primary's volume and take down the write path. PostgreSQL 13 and later bound this with a maximum slot retention size, and the CNPG documentation recommends setting it explicitly whenever slots are enabled [14]. Exceeding that bound does not merely delay the standby: the slot is invalidated and the standby can no longer resume, so it must be re-cloned in full. The conclusion is that the real ceiling on elastic scale-out is neither compute nor network but WAL retention capacity, and that this ceiling is a policy decision rather than a technical limit.

5.4 Vertical scaling and rolling updates

Changing Pod resource requests or limits is a different operation with a different failure profile. CNPG performs a rolling update in which replicas are replaced first, in descending serial order, and the primary is replaced last. Data is not re-cloned, because the same claims are reused, so the operation is bounded by shutdown, scheduling, and startup rather than by data volume [19]. Image changes, configuration changes that require a restart, and instance-manager changes after an operator upgrade follow the same machinery.

The primary is handled according to a configurable strategy. In the default unsupervised mode the operator either restarts the primary in place or, failing that, switches over to the most advanced standby. In supervised mode the rollout pauses after the replicas and waits for an explicit promotion or restart. The choice trades recovery time against recovery point: an in-place restart may have to pull an image after the write path is already unavailable, whereas a switchover promotes an instance that already runs the target specification at the cost of a brief write interruption. The timeout governing the graceful shutdown of the outgoing primary defaults to one hour and is therefore a bound on the worst case rather than an expected duration [21].

This is where single-instance deployments diverge sharply from multi-instance ones. With one instance there is no candidate to promote, so a vertical change is an in-place restart of the only primary and is disruptive by construction. Vertical elasticity without downtime therefore requires at least two instances, which in turn requires storage that both instances can attach. Resource management and storage are not separable concerns.

CNPG also documents that per-Pod autoscaling of either kind is a poor fit. A Vertical Pod Autoscaler cannot evict the primary while a PodDisruptionBudget protects it, so it stalls rather than converging, and the documented remedy is to apply its recommendations to the Cluster specification by hand [17].

5.5 Storage elasticity as the binding constraint

CNPG's own control over storage is narrow. It creates and owns a claim group per instance, it supports a separate volume for WAL, it treats the declared size as monotonic, and it exposes a flag controlling whether existing claims are resized along with the specification [21]. Everything below that level, including whether a claim can be resized at all, whether the resize is performed online, and whether the filesystem is expanded by the driver, belongs to the StorageClass and its CSI plugin [24].

This produces a structural asymmetry that the rest of the operator's design does not share. Compute elasticity is a first-class concern implemented in the reconciliation loop; storage elasticity is delegated entirely, and there is no fallback when the provisioner cannot deliver it. A claim whose resize is accepted at the API level but never satisfied by the driver leaves the cluster reporting a requested size that does not match the filesystem, and no amount of reconciliation resolves the discrepancy. A cluster that appears elastic in its specification can therefore be rigid in reality. Any elasticity claim about a CNPG deployment is, strictly, a claim about its storage provider.

Table 1: Cost profile of the elastic operations available in CNPG.

Operation	Asymptotic cost	Dominant factor	Blocking precondition
Horizontal scale-out	$O(\|\text{PGDATA}\|)$	database size, link bandwidth	storage able to host another full copy
Replica catch-up	$O(\text{applied lag})$	WAL retention bound	slot not invalidated
Vertical scale-up	$O(\text{restart})$	instance count ≥ 2	a standby able to take over
Volume growth	$O(\Delta\text{size})$	CSI expansion support	expandable StorageClass

Table 1 summarises the asymmetry. Three of the four operations are bounded by the operator's own state machine; the fourth is bounded by infrastructure the operator does not control. This is the basis for the argument in Section 8 that the choice of CSI backend, rather than any Cluster setting, determines the practical elasticity of a CNPG deployment.

6 Failure handling and failover

6.1 Failover sequence

Failover has two separate outcomes. First, a standby must be promoted to primary. Second, applications must reconnect through a route that identifies the new primary. A successful database promotion without client reconnection is still an application outage.

When the primary becomes unavailable, the operator uses Kubernetes health information and PostgreSQL state to select and promote a suitable standby. The precise behavior depends on the configured failover and switchover settings, the health of the Kubernetes control plane, the availability of replicas, and the amount of unapplied WAL. The read-write Service is then updated to target the promoted instance [3, 2].

CNPG's model does not require Patroni, etcd, ZooKeeper, or Consul as an additional failover manager. This reduces the number of coordination systems that operators must deploy and monitor. It also means that Kubernetes API availability and correctness are part of the database's failover path. A Kubernetes control-plane outage can prevent reconciliation or delay a routing update even when the PostgreSQL processes and storage remain healthy.

6.2 Loss of every instance

The case in which the primary and all standbys are lost simultaneously deserves separate treatment, because it is the one in which no promotion can occur and the operator's election logic rather than PostgreSQL's replication state determines the outcome. It is also the case that most often exposes an untested configuration.

When every instance stops without a clean shutdown, no instance restarts as a candidate primary. Each one replays its local log from the checkpoint recorded in its control file to the end of what it has received, so the recovery cost is proportional to the interval since the last checkpoint rather than to database size. Because all instances perform this work concurrently and on the same underlying hardware, this is the worst case for recovery time, not the average one. Only after that recovery does the operator compare the surviving states and name a primary. If the former primary returns with the highest position it simply resumes and no timeline change occurs; otherwise the most advanced standby is promoted and the timeline advances. The demoted former primary then has to be realigned. CNPG does this automatically

using PostgreSQL's rewind utility, which is available for this purpose because the operator enables hint logging in the write-ahead log by default [20, 23]. Where a rewind is impossible, the instance is re-cloned in full by the join path described in Section 5.2, which reintroduces the $O(|\text{PGDATA}|)$ cost at the worst possible moment.

The recovery point of this scenario is set entirely by the replication mode. With asynchronous replication, which is the default and the norm for these deployments, the promoted standby may lag the former primary, and transactions committed on the primary but not yet received by the promoted instance are lost. The CNPG documentation states the position without qualification: when synchronous replication is not enabled, some data loss is expected and accepted at failover, because a replica may lag behind the primary when promoted [15]. Raising the guarantee requires synchronous replication and, for the strongest form, quorum-based verification of the promotion itself.

A distinct and more severe case arises when the storage is lost with the instances. If the local log directory is destroyed, no amount of instance-level recovery helps: the cluster can only be rebuilt from the most recent base backup and the archived log. The recovery point then becomes a function of archiving cadence rather than of replication lag. Because CNPG forces a log switch every five minutes by default and ships each completed segment, the exposure is bounded by that interval for a healthy archive [21]. Where archiving has silently failed, the exposure reverts to the age of the last base backup, which is typically an order of magnitude larger. The practical distinction is that streaming replication and log archiving protect different failure scopes, and only the second protects against loss of the primary's storage.

6.3 Preventing divergent timelines: fencing and quorum

The dangerous variant of failure is not the loss of the primary but the loss of the operator's ability to reach it. A primary that remains running while isolated from the control plane will continue to accept writes, and if a new primary is promoted during that isolation the two will have diverged by the time the partition heals.

CNPG addresses the ordinary case structurally with a two-phase failover. The operator first marks the target primary as pending, which forces the outgoing primary to shut down so that its log senders stop in an orderly way; only then does it hold the election and promote. The shutdown itself is a graceful stop bounded by a configurable timeout, escalating to an immediate stop if the bound is exceeded [15]. That timeout is the explicit knob that trades recovery point against recovery time, and it is the parameter an operator should tune deliberately rather than accept by default.

For the residual case, in which the primary cannot be reached at all, CNPG provides two mechanisms. Fencing is a declarative annotation that guarantees the PostgreSQL process is shut down while the Pod is kept alive for inspection, and it can be applied to a single instance, a list, or the whole cluster [16]. It is the correct tool before any manual intervention on an instance whose state is uncertain. Failover quorum, introduced as an experimental feature in operator version 1.27, applies the same reasoning that underlies Dynamo-style quorum replication: promotion is permitted only when the set of reachable promotable replicas intersects every possible set of replicas that could have acknowledged a synchronous commit, that is, when $R + W > N$ [15, 29, 30]. Where the condition fails the operator declines to promote, preferring unavailability to silent divergence [34].

The consequence is easy to overlook and follows directly from the arithmetic. Quorum-based promotion has meaning only in conjunction with synchronous replication, so a deployment that leaves synchronous replication at its default gains no protection from enabling the annotation. In a two-instance cluster with one synchronous standby, loss of the primary leaves one promotable replica, and the condition $R + W > N$ fails: the operator will not promote. Two instances are therefore sufficient for non-disruptive maintenance but not for a durability guarantee under unplanned failure. This is developed further in Section 7.

6.4 Switchover and testing

A planned switchover is different from an unplanned failover. A switchover can drain or stop the current primary in a controlled manner and should reduce the risk of data loss. An unplanned failover must infer that the primary is unavailable and choose a surviving instance under less favorable conditions.

Organizations should test both paths. A useful drill records failure-detection time, promotion time, Service update time, connection-pool recovery time, and application recovery time. It should also verify a known set of committed writes on the new primary. Tests can include Pod deletion, node drain, node loss, stalled replication, and loss of an availability zone. They should use the same storage, network policy, connection pool, and monitoring configuration as production.

7 Topology and failure domains

Everything above describes mechanisms. Whether those mechanisms produce availability depends on a property the operator cannot create: independent failure domains. This section treats the two topologies a reader is most likely to encounter, and argues that the difference between them is qualitative rather than a matter of degree.

7.1 Multi-node deployments

In a multi-node deployment the instances are intended to occupy distinct nodes, so that loss of a node removes one instance rather than the cluster. CNPG enables Pod anti-affinity by default, which is the mechanism that expresses this intent, but the default rule is a preference rather than a requirement. Under resource pressure a preferred rule is simply not satisfied, and two instances may be co-located while the cluster continues to report healthy streaming replicas. Strengthening the rule to a requirement is therefore necessary for any availability claim, with the documented caveat that instances may then remain unscheduled until a suitable node exists [21]. The topology key also defaults to the host name, so protection against loss of an availability zone requires an explicit change of key rather than an increase in instance count.

The instance count interacts with the replication mode more sharply than is commonly assumed. A validating webhook refuses a scale-in that would take the count below the synchronous floor, which prevents a configuration from becoming silently unsatisfiable [17]. Less obviously, three instances is the smallest topology in which quorum-based promotion has any content, for the reason given in Section 6.3. A deployment of two instances therefore buys non-disruptive maintenance and a read replica, and it should not be described as providing a durability guarantee. This is a frequent and consequential misreading of what redundancy counts mean.

The storage choice must be aligned with the topology. Node-local volumes keep failure domains genuinely independent and give the most predictable latency, but they bind each instance irrecoverably to its node, so node loss requires re-cloning rather than rescheduling. Network-attached volumes restore mobility at the cost of added latency on every synchronous write and, more subtly, of coupling instances to a shared storage failure domain that the operator cannot observe. Neither arrangement is universally correct; the mistake is to assume that a distributed block layer makes a single-instance deployment highly available, which it does not, because replication within one failure domain replicates the failure as well as the data [28].

7.2 Single-node deployments

A single-node deployment deserves a precise description, because it is often either dismissed entirely or credited with properties it does not have.

What it does provide is real. Replicas give redundancy at the instance and process level, so a crashed server process, a corrupted instance, or a failed container can be replaced without recourse to backup. They permit non-disruptive minor upgrades and non-disruptive vertical changes by switchover, which is the single largest operational benefit available on one node. They provide an additional read endpoint for analytic traffic and a local source for re-cloning. And they shorten recovery from data-directory corruption, because a healthy second copy exists on the same host.

What it cannot provide is availability. Every instance shares one kernel, one storage controller, one power supply, and one network path. A node-level fault removes all of them at once, so redundancy reduces mean time to repair but leaves mean time between failures and unavailability unchanged. The default preferred anti-affinity rule cannot be satisfied, which means the shipped configuration places all instances on the node and reports them as streaming and healthy: the cluster status describes a high-availability topology that the placement does not implement. Any claim about this deployment must be phrased in terms of recovery, not resilience.

The single-node case also changes where the recovery point lives. Because there is no separate host to survive the loss of the node, the only protection against loss of the primary's storage is the offsite log archive. The archiving cadence, not the replication lag, becomes the operative recovery point parameter. This inverts the usual advice: on a multi-node deployment it is reasonable to treat archiving as a secondary protection behind streaming replication, whereas on a single node it is the primary protection and should be monitored as such.

Finally, storage elasticity on one node is constrained differently. Node-local provisioners bind claims to the node and typically expose neither expansion nor snapshots, whereas a distributed layer that does expose them derives no durability benefit because every replica lands on the same device. Volume growth on a single node is therefore a migration problem, solved either by pre-provisioning well beyond the expected requirement or by adopting a node-local provisioner that supports expansion and snapshots over a thin pool, as discussed in Section 8.

Table 2: Failure scopes addressed by each protection layer in a CNPG deployment.

Failure scope	Streaming replica	Block replication	Log archive	Snapshot
Server process or instance	yes	no	no	no
Pod or container	yes	no	no	no
Node	if separate node	yes	partial	no
Cluster-wide corruption	no	no	yes	yes
Site or region	no	no	yes	no
Control-plane outage	no	no	no	no

Table 2 makes the layering explicit. No single mechanism covers every scope, the archive is the only one that covers site loss, and none of them covers loss of the control plane, which remains the operator's structural dependency.

8 Storage backends and CSI elasticity

The preceding sections reduce the storage question to a small set of requirements. A backend suitable for a CNPG workload must permit online volume growth; it must keep synchronous write latency low, because PostgreSQL commits are bounded by log flush latency; it must present an exclusive read-write block device, since a data directory must never be shared at the

filesystem level; it should expose CSI snapshots, which convert scale-out from a full copy into a metadata operation; and its failure domains should align with the Pod anti-affinity rules rather than cut across them [24, 25].

8.1 Where redundancy should live

Beneath these requirements lies a design question that the literature treats explicitly: at which layer should redundancy be placed. One answer, exemplified by Aurora, is to move replication out of the database engine and into a purpose-built storage service, leaving a single writer to interact with a quorum of storage nodes [31]. The other answer, which CNPG adopts, is to keep redundancy in PostgreSQL's own streaming replication and to keep the storage layer simple. Both are coherent designs. What is not coherent is to stack many block replicas beneath many database replicas.

The cost of doing so is threefold. The write path pays for replication twice, and the classical analysis of replication overhead shows that this cost is not merely bandwidth but a multiplication of the durability protocol on the critical commit path [28]. The failure domains stop being independent in a way the operator can reason about, because a storage-layer rebuild triggered by one node's return is an event the database layer sees only as latency. And that latency is not benign: CNPG's default log sender and receiver timeouts are measured in seconds, so a storage rebuild that stalls input and output for longer than that will cause the primary to terminate replication, after which the standby must reconnect and may fall far enough behind to require re-cloning. The interaction between an asynchronous background repair mechanism and a synchronous replication timeout is the specific hazard, and it argues for treating block replication and database replication as alternatives rather than complements.

This reasoning is not confined to the database community. The best-practice guidance for at least one widely used distributed block storage system recommends, for applications that already replicate their own data, a single block replica with strict local data locality, explicitly to avoid the space and input-output overhead of volume replication [26, 27]. Applied to CNPG, that guidance converges on an unexpected result: the correct configuration of a replicated block layer beneath a replicated database is effectively local storage with an application programming interface, snapshots, and online resize.

8.2 Snapshot-backed cloning

The largest single elasticity gain available from the storage layer is not throughput but the cost of creating an instance. As established in Section 5.2, the join path copies the entire data directory. If the backend can produce a consistent point-in-time copy cheaply, that copy can replace the transfer, reducing the asymptotic cost of scale-out from the size of the database to the size of the metadata describing it. CNPG supports this both for backup and for the recovery path that a new instance follows [18, 25]. The trade-off is a stronger coupling to the storage vendor's snapshot semantics, and a consistency requirement that the snapshot be taken with the database's cooperation rather than by the storage system alone.

8.3 Recommendation

The phrase "maximum elasticity" is underdetermined until the constraint being relaxed is named, and Table 3 should be read with that in mind. Four conclusions follow from the analysis rather than from any particular product's specification.

First, elasticity is a property of the pair formed by a backend and a topology, not of the backend alone. The same distributed block system provides one durability profile across three nodes and an entirely different one on a single node, where every replica shares a device. Comparisons that ignore the topology are not meaningful.

Table 3: CSI-backed storage options evaluated against the requirements of a CNPG workload.

Backend class	Online resize	Write latency	Snapshot	Mobility	Principal limitation
Node-local host path	no	lowest	no	no	claim bound to node; no growth
Local thin pool (LVM)	yes	lowest	yes	no	single failure domain
Cloud block storage	yes	low	yes	in zone	single-zone attachment
Distributed block, replicated	yes	moderate	yes	yes	write amplification; repair latency tails
Distributed block, single replica	yes	low	yes	re-clone	no block-level redundancy
NVMe-over-fabric	yes	low	yes	yes	high infrastructure floor
DRBD-based	yes	low	yes	yes	operational complexity

Second, on a constrained topology the best available answer is a node-local provisioner over a thin pool. It is the only class that combines online expansion, snapshot support, and the lowest achievable write latency, and its limitation, the absence of node mobility, is a limitation the operator already handles through re-cloning. It offers the highest elasticity per unit of operational effort rather than the highest elasticity in the abstract.

Third, where multiple nodes and a genuine durability requirement coexist, the consistent choice is data replication at exactly one layer. Either the database replicates and the storage is local or single-replica, or the storage replicates and the database runs a single writer in the manner of Aurora. Deploying both is a decision to pay twice for redundancy while acquiring a second, less observable failure domain. The general-purpose distributed storage systems of the Ceph class are attractive on grounds of maturity and tooling, but their replication model is designed for a mixed workload rather than for the commit path of a transactional database [35]. Among replicated backends, the ones that expose a block device over a low-latency fabric rather than a general-purpose network file protocol are the ones whose latency behaviour is compatible with a synchronous commit path, and this is consistent with the broader finding that network latency dominates the cost of synchronous database replication [32].

Fourth, the analysis suggests a general principle for evaluating elasticity claims in operator-managed systems. Elasticity decomposes into independent axes: capacity, compute, redundancy, and mobility. Each axis is delivered by a different component, and each has a different failure mode. An operator can implement compute elasticity completely within its own reconciliation loop; it cannot implement capacity elasticity at all without delegating to a storage driver. Systems that appear uniformly elastic in their declarative interface are therefore frequently elastic along one axis and rigid along another, and the interface conceals which is which.

9 Performance and resource management

PostgreSQL performance in Kubernetes depends on the combined behavior of CPU, memory, storage, network, and scheduling. CNPG exposes Pod resource settings, but it cannot infer suitable values from the database schema or workload. Administrators should set requests and limits deliberately and tune PostgreSQL memory parameters within those bounds.

Storage is often the dominant factor. Local volumes may provide predictable latency and

high throughput, but they require a recovery design for node loss. Network-attached volumes simplify mobility but can have variable latency and throughput. Before deployment, ‘fio‘ can characterize storage and ‘pgbench‘ can provide a repeatable database workload baseline. The baseline should include latency, throughput, checkpoint behavior, WAL volume, and replica lag rather than only transactions per second.

Separating WAL and data storage can reduce I/O contention where the platform supports it. The benefit depends on the storage devices and workload, so it should be measured rather than assumed. Capacity planning must include data growth, temporary files, WAL retention during replica lag, backups, and the space needed for recovery operations.

Read replicas can serve reporting or read-heavy workloads, but they do not provide arbitrary linear read scaling. Queries routed to a standby observe a replication-defined consistency point and may be delayed. Synchronous replication is primarily a durability decision; using it for every read replica can add write latency without solving application-level consistency requirements.

10 Backups and disaster recovery

Replication is not a backup. A replicated deletion, corruption event, or incorrect transaction can reach every synchronous or asynchronous replica. CNPG supports PostgreSQL backup and recovery workflows using WAL archiving and object storage through its backup mechanisms and plugins. A recovery plan should define where base backups and WAL are stored, how long they are retained, who can restore them, and how restore success is verified [4, 5].

Point-in-time recovery (PITR) provides a way to recover to a time before an operational mistake, provided that the required base backup and WAL are available. Cross-zone or cross-region recovery requires more than an object-store bucket: it requires credentials, network access, compatible images, documented bootstrap configuration, and a tested process for promoting the recovered cluster.

CNPG can support passive or read-only PostgreSQL clusters in other Kubernetes clusters through declarative configuration. The local operator does not provide automatic cross-Kubernetes-cluster failover by itself. A higher-level management system or a documented human procedure must coordinate that decision [2].

11 Security

Security is a property of the complete deployment, not only of the operator image. CNPG supports TLS for client connections and replication, certificate management, custom certificate authorities, and integration with cert-manager. PostgreSQL authentication can also integrate with enterprise mechanisms such as LDAP where configured and supported by the deployment [6, 7].

The Kubernetes configuration should restrict access to the Cluster CR, Secrets, PVCs, and operator resources using least-privilege RBAC. NetworkPolicies should limit database traffic to intended application and replication paths. Backup credentials should be scoped to the required object-store paths, and the object store should enforce encryption, retention, and access logging appropriate to the data.

CNPG writes structured logs to standard output, which allows Kubernetes logging systems to collect database and operator events centrally. Centralized logs should be treated as security-relevant records because they can reveal authentication failures, topology changes, recovery activity, or misconfigured access. The project also documents security practices and publishes its source and release artifacts for review [8].

12 Upgrades and lifecycle management

Operator upgrades and PostgreSQL upgrades are related but distinct. The operator can be upgraded independently of the managed PostgreSQL workload, while a PostgreSQL image change may cause a rolling update, switchover, or require an offline migration depending on the version transition and configuration.

Minor PostgreSQL upgrades can generally use a rolling process because physical replicas can be replaced while another instance remains available. Major upgrades require a different procedure because PostgreSQL does not support in-place data-directory compatibility across major versions. CNPG provides declarative workflows for supported upgrade and import paths, but the application must still be prepared for connection interruptions and the operator must have enough healthy replicas to preserve availability [9].

An upgrade plan should include a tested backup, compatibility checks for extensions, a rollback or restore path, a maintenance communication plan, and post-upgrade validation. Configuration changes that alter Pod templates may trigger a rollout, while other metadata changes may require an explicit restart. Operators should check the relevant version documentation instead of assuming that every manifest change has the same lifecycle effect.

13 Observability and governance

Useful monitoring covers both the PostgreSQL data plane and the Kubernetes control plane. Data-plane indicators include primary and standby role, replication lag, WAL generation, checkpoint behavior, database connections, transaction latency, disk utilization, and backup age. Control-plane indicators include operator health, reconciliation errors, Pod readiness, PVC attachment, scheduling failures, Service endpoints, and certificate expiry.

Alerts should lead to a runbook. For example, a replication-lag alert should identify whether the cause is I/O pressure, network loss, a long-running transaction, insufficient resources, or a backup or recovery operation. A progressing cluster condition should be investigated through events and resource relationships rather than cleared by deleting status or manually removing a resource.

Governance connects technical settings to business requirements. The cluster specification should document the intended RPO, RTO, availability zones, backup retention, maintenance window, owner, and escalation path. Regular recovery exercises provide evidence that the stated objectives are achievable. Without such exercises, a configured backup or failover mechanism is only an assumption.

14 Advantages and limitations

CNPG's principal advantage is a coherent Kubernetes-native control model. Teams do not need to operate a separate failover manager for the local cluster, and the desired topology can be reviewed and reconciled as Kubernetes configuration. Direct Pod and PVC management lets CNPG make leadership and lifecycle decisions based on database state rather than Pod ordinals. Native PostgreSQL replication, Services, backups, certificates, and metrics fit familiar Kubernetes workflows.

The same design creates limits. CNPG depends on Kubernetes API and scheduling behavior for orchestration, so control-plane incidents affect database management and may affect failover completion. The operator cannot compensate for poor storage, a partitioned network, insufficient capacity, or an application that never reconnects. Cross-cluster disaster recovery needs a higher-level authority. Finally, teams moving from Patroni-based or managed PostgreSQL systems must learn new commands, status conditions, runbooks, and assumptions about where high-availability decisions are made.

There is no universal best topology. A single Kubernetes cluster with replicas across zones provides local high availability, while an independent recovery cluster provides a different failure boundary. Synchronous replication protects specific commit paths but can reduce availability when the synchronous member is unavailable. Asynchronous replicas preserve write availability but may lose recent transactions. CNPG makes these choices configurable; it does not eliminate the underlying trade-offs.

The analysis in Sections 5 to 8 sharpens this into a structural observation. The operator implements compute elasticity within its own reconciliation loop and does so completely. It cannot implement capacity elasticity at all, because volume growth is delegated to a CSI driver that it does not observe. And it cannot manufacture availability, because that is a property of the failure domains it is given, not of the instances it creates. The declarative interface presents all three as similarly configurable, which is precisely what makes the asymmetry easy to miss. A deployment that is elastic in its specification may still be rigid on disk and fragile on node loss.

15 Discussion

CNPG demonstrates that a database operator can move much of the high-availability control logic into Kubernetes without treating PostgreSQL as a stateless workload. The instance manager handles process-level behavior, PostgreSQL handles replication and recovery, and the operator coordinates resources and topology. This division is simpler than a stack of independent controllers only when the organization already understands Kubernetes and can operate its control plane reliably.

The practical lesson is that declarative configuration does not mean hands-off administration. It shifts administration toward specifying policy, observing reconciliation, and testing the result. Scaling requires storage and replication capacity. Failover requires both promotion and client reconnection. Backups require restoration tests. Security requires Kubernetes, PostgreSQL, certificate, and object-store controls. Upgrades require version-specific validation. These responsibilities remain even when the operator automates the ordinary path.

The model also changes how failure should be investigated. A database incident may begin with PostgreSQL, but the visible symptom may be a Kubernetes Service, a PVC attachment, a node condition, an API timeout, or a connection pool. Effective diagnosis therefore needs a joint view of Kubernetes events, operator status, instance-manager logs, PostgreSQL logs, replication state, and application telemetry.

16 Conclusion

CloudNativePG provides a practical Kubernetes-native model for operating PostgreSQL clusters with declarative configuration, native replication, automated failover, self-healing, backups, and lifecycle automation. Its direct management of Pods and PVCs, together with the in-container instance manager, avoids making StatefulSet ordinals or a separate HA framework responsible for PostgreSQL leadership.

The benefits are strongest when the surrounding platform is well designed. Replicas must be distributed across suitable failure domains, storage must be benchmarked and recoverable, clients must use role-aware Services, backups must be restored in practice, and failover must be rehearsed. CNPG reduces operational complexity, but it does not remove the need for database expertise or control-plane reliability. For Kubernetes teams that accept those responsibilities, CNPG offers a coherent way to place PostgreSQL lifecycle management under a single declarative operating model.

Three findings warrant restating because they apply to operator-managed stateful systems generally and not only to this one. Elasticity decomposes into independent axes whose costs

differ by orders of magnitude: scale-out is linear in data volume because the join is a cold full copy, whereas scale-up is bounded by a restart. Write-ahead log retention capacity, not compute or bandwidth, is the practical ceiling on how far a cluster can drift before an instance must be rebuilt. And availability cannot be delegated to an operator at all, because a preferred anti-affinity rule on a single node describes a topology that does not exist, and a healthy cluster status reports the declared intent rather than the achieved placement. These are properties of the design rather than defects in it, but they determine what a deployment can actually be relied upon to do.

References

[1] CloudNativePG, "CloudNativePG: PostgreSQL Operator for Kubernetes," project website. Available at: https://cloudnative-pg.io/.

[2] CloudNativePG, "Architecture," documentation. Available at: https://cloudnative-pg.io/documentation/current/architecture/.

[3] CloudNativePG, "Postgres instance manager," documentation. Available at: https://cloudnative-pg.io/documentation/current/instance_manager/.

[4] CloudNativePG, "Backup," documentation. Available at: https://cloudnative-pg.io/documentation/current/backup/.

[5] CloudNativePG, "Barman Cloud plugin," documentation. Available at: https://cloudnative-pg.io/documentation/current/recovery/.

[6] CloudNativePG, "Certificates," documentation. Available at: https://cloudnative-pg.io/documentation/current/certificates/.

[7] CloudNativePG, "Authentication," documentation. Available at: https://cloudnative-pg.io/documentation/current/ssl_connections/.

[8] CloudNativePG, "Security," project documentation and repository security material. Available at: https://github.com/cloudnative-pg/cloudnative-pg/security.

[9] CloudNativePG, "Upgrades," documentation. Available at: https://cloudnative-pg.io/documentation/current/upgrade/.

[10] Kubernetes Authors, "StatefulSets," Kubernetes documentation. Available at: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/.

[11] PostgreSQL Global Development Group, "High Availability, Load Balancing, and Replication," PostgreSQL documentation. Available at: https://www.postgresql.org/docs/current/high-availability.html.

[12] PostgreSQL Global Development Group, "Write-Ahead Logging," PostgreSQL documentation. Available at: https://www.postgresql.org/docs/current/wal.html.

[13] CloudNativePG, "CloudNativePG source repository," GitHub. Available at: https://github.com/cloudnative-pg/cloudnative-pg.

[14] CloudNativePG, "Replication," documentation, version 1.27. Available at: https://cloudnative-pg.io/docs/1.27/replication/.

[15] CloudNativePG, "Automated failover," documentation, version 1.27. Available at: https://cloudnative-pg.io/docs/1.27/failover/.

[16] CloudNativePG, “Fencing,” documentation, version 1.27. Available at: https://cloudnative-pg.io/docs/1.27/fencing/.

[17] CloudNativePG, “Resource management,” documentation, version 1.27. Available at: https://cloudnative-pg.io/docs/1.27/resource_management/.

[18] CloudNativePG, “Storage,” documentation, version 1.27. Available at: https://cloudnative-pg.io/docs/1.27/storage/.

[19] CloudNativePG, “Rolling updates,” documentation, version 1.27. Available at: https://cloudnative-pg.io/docs/1.27/rolling_update/.

[20] CloudNativePG, “Frequently asked questions,” documentation, version 1.27. Available at: https://cloudnative-pg.io/docs/1.27/faq/.

[21] CloudNativePG, “API reference: Cluster and AffinityConfiguration,” documentation, version 1.27, and the corresponding type definitions in `api/v1/cluster_types.go`.

[22] PostgreSQL Global Development Group, “pg_basebackup,” PostgreSQL documentation. Available at: https://www.postgresql.org/docs/current/app-pgbasebackup.html.

[23] PostgreSQL Global Development Group, “pg_rewind,” PostgreSQL documentation. Available at: https://www.postgresql.org/docs/current/app-pgrewind.html.

[24] Kubernetes Authors, “Container Storage Interface (CSI) drivers,” Kubernetes documentation. Available at: https://kubernetes-csi.github.io/docs/.

[25] Kubernetes Authors, “Volume Snapshots,” Kubernetes documentation. Available at: https://kubernetes.io/docs/concepts/storage/volume-snapshots/.

[26] Longhorn, “Best Practices,” documentation, version 1.9. Available at: https://longhorn.io/docs/1.9.0/best-practices/.

[27] Longhorn, “Volume Expansion,” documentation, version 1.9. Online expansion available since version 1.4.0. Available at: https://longhorn.io/docs/1.9.0/nodes-and-volumes/volumes/expansion/.

[28] J. Gray, P. Helland, P. O’Neil, and D. Shasha, “The dangers of replication and a solution,” in *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, 1996, pp. 173–182.

[29] G. DeCandia, D. Hastorun, M. Jampani, et al., “Dynamo: Amazon’s highly available key-value store,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007, pp. 205–220.

[30] H. Attiya, A. Bar-Noy, and D. Dolev, “Sharing memory robustly in message-passing systems,” *Journal of the ACM*, vol. 42, no. 1, pp. 124–142, 1995.

[31] A. Verbitski, A. Gupta, D. Saha, et al., “Amazon Aurora: Design considerations for high throughput cloud-native relational databases,” in *Proceedings of the 2017 ACM SIGMOD International Conference on Management of Data*, 2017, pp. 1041–1052.

[32] E. Zamanian, X. Yu, M. Stonebraker, and T. Kraska, “Rethinking database high availability with RDMA networks,” *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1634–1647, 2019.

[33] C. Curino, E. Jones, R. Popa, et al., "Relational Cloud: A database-as-a-service for the cloud," in *Proceedings of the 5th Biennial Conference on Innovative Data Systems Research (CIDR)*, 2011.

[34] P. Bailis, A. Davidson, A. Fekete, A. Ghodsi, J. M. Hellerstein, and I. Stoica, "Highly available transactions: Virtues and limitations," *Proceedings of the VLDB Endowment*, vol. 7, no. 3, pp. 181–192, 2013.

[35] S. A. Weil, S. A. Brandt, E. L. Miller, D. D. E. Long, and C. Maltzahn, "Ceph: A scalable, high-performance distributed file system," in *Proceedings of the 7th Symposium on Operating Systems Design and Implementation (OSDI)*, 2006, pp. 307–320.